“Linux Signal-Based Process Communication and Control System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Operating Systems and Systems Programming (25CS2104E) · 2026–27, Term-I · Section 10 · Team 06

By

Name	Roll Number
R. Radhakrishna	2520090063
R. Jagadeesh	2520090030
P. Vedadri Naidu	2520090060

Under the Esteemed Guidance of

Mr. Raghupathi Manthena, Assistant Professor Department of Computer Science and Engineering

K L (Deemed to be) University – Koneru Lakshmaiah Education Foundation (Deemed to be University estd. u/s. 3 of the UGC Act, 1956) Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Declaration

K L (Deemed to be) University DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

The Project Report entitled **“Linux Signal-Based Process Communication and Control System”** is a record of bonafide work of **R. Radhakrishna - 2520090063, R. Jagadeesh - 2520090030, P. Vedadri Naidu - 2520090060** submitted in partial fulfillment for the award

of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Name	Roll Number
R. Radhakrishna	2520090063
R. Jagadeesh	2520090030
P. Vedadri Naidu	2520090060

Certificate

K L (Deemed to be) University DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

This is to certify that the project report entitled **“Linux Signal-Based Process Communication and Control System”** is a bonafide work done and submitted by **R. Radhakrishna - 2520090063, R. Jagadeesh - 2520090030, P. Vedadri Naidu - 2520090060** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide	Signature of the Course Coordinator	Signature of the HOD

Acknowledgement

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or another in the completion of our project.

Our greatest appreciation goes to our Course Coordinator Dr. K. Anusha and our guide Mr. Raghupathi Manthena, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to Dr. P. Venkateshwara Rao (CSE), Head of the Department of Computer Science Engineering, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

R. Radhakrishna 2520090063 R. Jagadeesh 2520090030 P. Vedadri Naidu 2520090060

Abstract

Modern systems consist of many cooperating processes, and efficient inter-process communication (IPC) is needed to coordinate their activities. The Linux Signal-Based Process Communication and Control System is a project that uses POSIX signals for lightweight, event-driven process synchronization and control. Traditional IPC mechanisms such as pipes, message queues and sockets are reliable but introduce overhead when only a simple notification (a status request, a timeout, a shutdown order) has to be delivered. Signals offer a low-latency alternative that the kernel delivers asynchronously to the target process.

The project implements a signal-handler framework that processes SIGUSR1, SIGUSR2, SIGALRM and SIGTERM to coordinate process states, manage resource monitoring and shut processes down gracefully. A master–slave architecture is used: the master process spawns slave worker processes with fork() and exec(), sends control signals to them with kill(), and collects their replies with sigqueue(). Signal masking with sigprocmask() and sigsuspend() prevents race conditions in critical sections, and only async-signal-safe operations are performed inside handlers.

A monitoring dashboard reads /proc/PID/stat to report CPU usage, memory consumption and per-slave event counters in real time. The system is designed to manage up to 100 concurrent slave processes. The programs are written in C, compiled with GCC and executed on Ubuntu Linux. Overall, the project gives practical insight into signal delivery semantics, signal masking, process state management and building efficient systems-level applications on Linux/POSIX platforms.

Index

S. No.	Chapter Topics / Subtopics
–	Acknowledgement · Abstract
1	**Introduction** – 1.1 Background of the Project · 1.2 Problem Statement · 1.3 Scope of the Project · 1.4 Objectives of the Project · 1.5 Importance of the Application · 1.6 Target Users / Audience
2	**OS Concepts and Background Study** – 2.1 Operating System Concepts Related to the Project · 2.2 Existing Methods / Algorithms · 2.3 Proposed Approach · 2.4 Advantages of the Proposed Approach
3	**System Requirements** – 3.1 Hardware Requirements · 3.2 Software Requirements · 3.3 Operating System Environment – Ubuntu / WSL · 3.4 Development Tools and Compilers
4	**Technology and Tools Used** – 4.1 C Programming · 4.2 GCC Compiler · 4.3 Linux System Calls · 4.4 Synchronization with Signals · 4.5 Linux Commands and Utilities · 4.6 Other Tools Used
5	**System Architecture and Design** – 5.1 Overall System Architecture · 5.2 System Workflow · 5.3 Flowchart / Data Flow · 5.4 Module Description · 5.5 Input and Output Design
6	**Implementation** – 6.1 Module-wise Implementation · 6.2 Algorithm / Pseudocode · 6.3 Core C Implementation · 6.4 Linux System Calls / APIs Used · 6.5 Process / Thread Implementation · 6.6 IPC / Synchronization Implementation · 6.7 Memory Management Implementation · 6.8 Optional User Interface Implementation

S. No.	Chapter Topics / Subtopics
7	**Execution and Demonstration** – 7.1 Compilation in Ubuntu · 7.2 Execution Commands · 7.3 Sample Inputs and Outputs · 7.4 Screenshots of Execution · 7.5 Demonstration of OS Concepts
8	**Conclusion** – 8.1 Summary · 8.2 Objectives Achieved · 8.3 Results Achieved · 8.4 Skills and Learning Outcomes
9	**References** – 9.1 Books · 9.2 Research Papers · 9.3 Linux / GNU Documentation · 9.4 Online Resources

---

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

A modern operating system runs many programs at the same time. Each running program is a process, and the kernel gives every process its own private virtual address space so that one process cannot read or overwrite the memory of another. This isolation is essential for security and stability, but real applications are rarely a single process: servers, shells, daemons and worker pools all need their processes to notify each other of events and to coordinate their states.

Linux provides many IPC mechanisms, including pipes, FIFOs, message queues, shared memory, semaphores, sockets and **signals**. A signal is a small software notification delivered asynchronously by the kernel to a process. Signals are the oldest UNIX IPC mechanism and are used for everything from terminating a program (Ctrl+C sends SIGINT) to telling a parent that a child has exited (SIGCHLD). Because a signal carries almost no data, it is extremely cheap, which makes it well suited to control and notification traffic.

This project, “Linux Signal-Based Process Communication and Control System”, studies and implements signal-based IPC in C using the POSIX signal API: sigaction(), sigprocmask(), sigsuspend(), kill(), sigqueue(), alarm(), fork(), exec() and waitpid(). The programs are developed and executed on Ubuntu Linux.

1.2 Problem Statement

In modern operating systems, efficient IPC is crucial for building robust and scalable applications. Traditional mechanisms such as pipes, message queues and sockets are reliable, but they introduce significant overhead for simple event notifications and process synchronization. Many systems need low-latency communication for control signals, status updates and process coordination. POSIX signals are powerful and lightweight, yet they are often under-used or misunderstood. Developers frequently face four challenges:

1. Implementing signal handlers that work reliably with multiple processes.
2. Managing signal masks and delivery mechanisms correctly.
3. Coordinating multiple processes with varying priorities and loads.
4. Monitoring process states in real time.

This project addresses these challenges by developing a signal-based IPC framework that demonstrates good practice in signal handling, process synchronization and resource management, and shows how signals can be used to build responsive, low-latency applications that scale to many concurrent processes.

1.3 Scope of the Project

- Implementation of a master process that spawns up to 100 slave worker processes using fork() and exec().
- Implementation of signal handlers for SIGUSR1 (status request), SIGUSR2 (shutdown), SIGALRM (periodic monitoring / timeout) and SIGTERM (graceful termination).
- Use of signal masks (sigprocmask(), sigsuspend()) to protect critical sections and prevent lost wake-ups.
- A monitoring dashboard that reads /proc/PID/stat to display CPU time, memory and signal statistics.
- Handling of edge cases: orphaned processes, signal coalescing and coordinated graceful shutdown.
- Execution and testing in the Linux terminal on Ubuntu using the GCC compiler. Other IPC mechanisms (pipes, FIFOs, message queues, sockets) are discussed only for comparison.

1.4 Objectives of the Project

1. Implement a robust signal-handling framework that correctly manages POSIX signals (SIGUSR1, SIGUSR2, SIGALRM, SIGTERM) with proper signal masking and delivery semantics.
2. Design and develop a master–slave process architecture in which the master coordinates multiple slave processes through signal-based communication protocols.
3. Create a process monitoring system that tracks CPU usage, memory consumption and signal delivery statistics in real time using the /proc filesystem and signal counters.
4. Demonstrate advanced OS concepts including signal atomicity, critical sections, async-signal-safe functions and race-condition prevention in signal handlers.

1.5 Importance of the Application

Signals are a core topic of Operating Systems and Systems Programming. Almost every non-trivial Linux program uses them, whether to shut down cleanly, reload configuration, handle timers or reap children. Understanding signal delivery, masking and handler restrictions gives a practical grasp of process states, asynchronous events, kernel–user transitions and race conditions – concepts that appear in shells, daemons, init systems, servers and job-control tools.

1.6 Target Users / Audience

- Students of Operating Systems and Systems Programming who want a working reference for signal-based IPC.
- Faculty and evaluators who need a clear demonstration of signal handling and process control.
- Beginner Linux/C programmers learning multi-process programming.
- Developers who need a lightweight event-notification mechanism between cooperating processes.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process and Process Management. A process is a program in execution. In Linux a new process is created with fork(), which duplicates the caller; the original is the parent and the copy is the child. fork() returns the child's PID to the parent and 0 to the child. A child

can then replace its program image with exec(). The parent collects the child's exit status with waitpid(), which prevents zombie processes. A running process moves between the states *running*, *ready*, *sleeping* and *stopped*; SIGSTOP and SIGCONT move a process into and out of the stopped state, which is how job control (Pause / Resume) works.

Signals. A signal is a software notification sent to a process by the kernel or by another process. Between the moment a signal is generated and the moment it is delivered it is *pending*. A process may *block* a signal using a signal mask, in which case it stays pending until unblocked. When delivered, the signal either runs the default action (terminate, ignore, stop, continue), is ignored, or invokes a user-defined *signal handler*.

Signal Handlers and Async-Signal Safety. A handler can interrupt the program at any instruction. If it calls a function that is not re-entrant (printf(), malloc()) while the main code is inside the same function, data can be corrupted. Therefore handlers must call only *async-signal-safe* functions and communicate with the main code through variables of type `volatile sig_atomic_t`. The usual pattern, used throughout this project, is: the handler sets a flag, and the main loop does the real work.

Signal Masking and Critical Sections. sigprocmask() blocks or unblocks signals. Blocking a signal around a critical section prevents a handler from running in the middle of it. sigsuspend() atomically replaces the mask and sleeps, which closes the classic race in which a signal arrives between "check the flag" and "call pause()".

Signal Coalescing. Standard signals are not queued: if SIGUSR1 is sent three times while it is blocked, it is delivered only once. Real-time signals (SIGRTMIN…SIGRTMAX) sent with sigqueue() are queued and can carry an integer or pointer value (`union sigval`). The project uses a real-time signal for slave-to-master status replies so that no report is lost.

Process Synchronization. Cooperating processes must execute in a correct order. Here synchronization is achieved by (i) signal delivery itself, (ii) sigsuspend() which sleeps with no busy waiting, (iii) SIGCHLD / waitpid() for termination, and (iv) alarm() for periodic timing.

/proc Filesystem. /proc is a virtual file system through which the kernel exposes process information. The file /proc/PID/stat contains, among others, the CPU time consumed in user mode (utime) and kernel mode (stime) and the resident set size (rss).

OS Concept / Linux API	Purpose in the Project
Process Management	Creation of slaves with fork()/exec(); waiting and reaping with waitpid().

OS Concept / Linux API	Purpose in the Project
Signals (IPC)	Event notification and control between master and slaves.
Signal Handlers – sigaction()	Reliable registration of handlers for SIGUSR1, SIGUSR2, SIGALRM, SIGTERM.
Signal Masks – sigprocmask(), sigsuspend()	Prevent race conditions in critical sections; sleep until a signal arrives.
kill() / sigqueue()	Send control signals (master → slave) and status replies with data (slave → master).
volatile sig_atomic_t	Signal-safe flags shared between handler and main code.
/proc filesystem	Real-time CPU and memory statistics of every slave.
Process States	SIGSTOP / SIGCONT for pause and resume; SIGTERM / SIGUSR2 for termination.

2.2 Existing Methods / Algorithms

Linux offers several IPC methods. Each has its own strengths and costs.

IPC Method	How it works	Limitation / Remark
Pipes	Unidirectional kernel buffer between related processes.	Only between processes with a common ancestor; byte stream, no event semantics.
FIFOs	Pipe with a name in the file system.	Blocks until both ends are open; one channel per pair.
Message Queues	Kernel-managed queue of typed messages.	Message size limits; more complex API; kernel memory use.
Shared Memory	Processes map the same memory region.	Fastest for data, but needs explicit synchronization.
Semaphores	Counters used to coordinate access to resources.	Provide synchronization only, not data transfer.

IPC Method	How it works	Limitation / Remark
Sockets	Endpoint for communication, local or networked.	Heavier setup; suited to client–server designs.
Regular Files	One process writes a file, another reads it.	Slow, needs polling, risk of race conditions.
Signals	Asynchronous notification sent by the kernel.	Almost no data; standard signals coalesce; handler restrictions.

2.3 Proposed Approach

The proposed approach implements a signal-driven master–slave system with three components: the **Master Process Controller**, the **Slave Worker Processes** and the **Monitoring Dashboard**.

- **Step 1 – Handler registration.** signal_setup() registers handlers with sigaction(). The master handles SIGUSR1 (print status now), SIGUSR2 / SIGTERM / SIGINT (shutdown), SIGALRM (periodic monitoring), SIGCHLD (a slave exited) and a real-time signal (status reply). Each slave handles SIGUSR1 (status request), SIGUSR2 / SIGTERM (shutdown) and SIGALRM (timeout).
- **Step 2 – Spawning.** The master calls fork() and exec() N times ($1 \leqslant N \leqslant 100$) to start the slaves.
- **Step 3 – Communication flow.** Master sends SIGUSR1 → slave receives and processes → slave updates its status and replies with sigqueue() → master collects the result.
- **Step 4 – Monitoring.** On every SIGALRM tick the master reads /proc/PID/stat of each slave and prints the dashboard.
- **Step 5 – Graceful shutdown.** On SIGTERM / SIGINT the master sends SIGUSR2 to every slave, waits for all of them with waitpid() and exits, leaving no orphans or zombies.
- **Step 6 – Testing and documentation.** The programs are compiled with GCC, tested with different process counts and signal patterns, and the results are captured as outputs and screenshots.

2.4 Advantages of the Proposed Approach

- **Lightweight** – a signal needs no buffer, descriptor or connection set-up, so notification latency is very low.

- **Event-driven** – processes sleep in sigsuspend() until something happens; no polling or busy waiting.
- **Kernel-managed delivery** – the kernel takes care of queuing, masking and delivery.
- **Safe** – handlers only set flags, so there are no unsafe calls inside handlers.
- **Scalable** – one master can supervise up to 100 slaves.
- **No extra libraries** – only standard POSIX/Linux headers are used.

CHAPTER 3 – SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The programs are lightweight and run on any modern computer.

Component	Minimum	Recommended
Processor	Any x86/x64, 1 GHz	Dual-core 2.0 GHz or higher
RAM	512 MB	2 GB or more
Storage	5 GB free disk space	10 GB or more
Display	Standard monitor with terminal access	Full HD monitor
Input devices	Keyboard and mouse	Keyboard and mouse

3.2 Software Requirements

Software	Details
Operating System	Linux kernel 3.0+ (Ubuntu 20.04 LTS or later / CentOS) or Windows with WSL2 running Ubuntu
Programming Language	C (C99/C11)
Compiler	GCC 4.8.5 or higher (Clang also supported)
C Library	glibc 2.17 or higher
Debugging / Profiling	GDB, Valgrind
Build Tool	GNU Make

Software	Details
Editor / IDE	VS Code, Vim or Emacs
Shell / Terminal	Bash
Version Control	Git and GitHub

3.3 Operating System Environment – Ubuntu / WSL

The project is developed on Ubuntu Linux, a Debian-based distribution that ships with the Linux kernel, the GNU C library (glibc) and the Bash shell. Signals, fork() and /proc are native kernel features, so no additional configuration is required.

Team members who use Windows can run the same code through the Windows Subsystem for Linux (WSL2), which provides a real Linux kernel. After installing Ubuntu from the Microsoft Store, the build tools can be installed with:

```
sudo apt update
sudo apt install build-essential gdb valgrind strace ltrace
```

3.4 Development Tools and Compilers

- **GCC / Clang** – compile the C source files into executables (`gcc file.c -o file`).
- **GNU Make** – automates compilation of master and slave (`make`).
- **GDB** – debugging of process behaviour.
- **Valgrind** – memory-error and leak detection.
- **strace / ltrace** – trace system calls to verify kill(), sigaction() and sigqueue() activity.
- **Git / GitHub** – version control and sharing of the source code among the team.